Lynn Sherman

Mathematician & Software Developer

M.S. Mathematics, University of Tennessee — Cryptography · AI Systems · Security Engineering

---

GitHub: github.com/lynekojawa **LinkedIn:** linkedin.com/in/lynn-sherman-92092a345

Overview

I am a mathematician and software engineer with an M.S. from the University of Tennessee, building at the intersection of cryptography, AI systems, and security engineering.

I teach mathematics, and I build. Since April, I have shipped six public projects spanning cryptographic communication tools, finite-field determinant calculator, LLM study infrastructure, repository visualization, and multi-agent orchestration frameworks. Everything is on GitHub.

My background spans cryptography, number theory, and partial differential equations. I bring mathematical reasoning to implementation, with a focus on understanding system behavior, identifying failure points, and designing reliable systems.

Technical Skills

Languages	Python, LaTeX, YAML/JSON
Databases	SQLite, Supabase, PostgreSQL, Firebase
Frameworks	Streamlit, PySide6
Tools	Git, Ollama, Pydantic, API integration
AI / Infrastructure	Local LLM inference, Context Engineering, Multi-Agent Orchestration, Prompt Engineering
Mathematics	Cryptography, Number Theory, Linear Algebra, Abstract Algebra, Discrete Mathematics, Partial Differential Equations, Stochastic Processes, Graph Theory

Languages

English Korean Japanese(Daily fluency)

Education

M.S. Mathematics University of Tennessee

Partial Differential Equations

Conference: Speaker, "Lost in Translation: Connections Between Music & Math", MAA Southeast Section Meeting Spring 2024

B.S. Mathematics *University of Mary Washington*

Cryptography · Number Theory

Honors thesis: ElGamal cryptography and finite field computation

Projects

Project-EVE v2.1 *Cryptography · Application Development · Message Integrity*

Full-stack encrypted messenger implementing the ElGamal encryption system with Diffie–Hellman key exchange and client-side key architecture.

What I built:

- ElGamal public-key encryption and Diffie–Hellman key exchange with local private-key handling
- SHA-256-based key derivation with domain separation
- Shared-Secret Cipher Identification (SSCI): the active cipher engine is masked using ElGamal shared secret (stored_header = Engine_ID XOR HASH(K)[0]), so an attacker cannot determine which of four cipher engines was used without the recipient's private key, forcing blind brute-force across all engines simultaneously.
- Cryptographic Integrity Layer (HMAC-SHA256): every message packet carries a 32-byte HMAC over the ciphertext; tampering at the database level triggers a signature mismatch before decryption begins, separating confidentiaty and integrity as independent guarantees.
- EVE Attacker Dashboard: simulation environment where an adversary can download encrypted messages by username and run four attack analyses: HMAC integrity tamper simulation, exhaustive Caesar shift brute-force, SSCI obfuscation trial analysis and frequency distribution analysis.

Why it exists: Applying cryptographic theory from my undergraduate ElGamal research to a practical, deployable system with explicit security boundaries.

Source: github.com/lynekojawa/Project-EVE

Project-Q *Local LLM · Spaced Repetition · Document Processing*

A local LLM-powered study pipeline designed to turn technical textbooks into structured summaries and spaced-repetition study sessions. Built to explore whether local inference could support technical learning without relying on hosted APIs or their token limits.

What I built:

- Implemented a multi-phase pipeline combining PDF text ingestion, custom text sanitization, a relational SQLite ledger, and SuperMemo-2 (SM-2) spaced-repetition scheduling.
- Built Pydantic schema contracts for structured LLM generation through Ollama, alongside a Streamlit skill-tree interface for prerequisite tracking and study-session management.
- Evaluated local LLM summary generation across 114 study nodes from Jeff Erickson's *Algorithms*. The pipeline produced partial summaries for 91 nodes, while the remaining nodes failed to generate summaries. Audited the resulting artifacts to identify reliability issues involving irrelevant content, chapter-boundary confusion, and PDF parsing errors.

Why it exists: I needed to accelerate algorithm study without token limits, API costs, or an internet dependency.

Source: github.com/lynekojawa/Project-Q

ATS-Simulator v2.1 *NLP · Hiring Systems · Text Processing*

Weighted keyword matcher that reverse-engineers ATS (Applicant Tracking System) resume scoring logic.

What I built:

- Implemented weighted keyword analysis with Jaccard similarity, missing-keyword identification, and experience requirement extraction from PDF and TXT documents.
- Developed a hybrid scoring engine that combines global resume-to-JD matching with zone-specific scoring against user-selected sections such as Qualifications and Duties.
- Built automatic job-description header detection and zone tracking, allowing users to prioritize high-signal sections without manually defining every boundary.

Why it exists: Frustrated by the opacity of automated hiring filters, I built my own tool to analyze and understand the hiring process.

Source: github.com/lynekojawa/ATS-Logic-Simulator

Project Trace *Graph Visualization · Software Architecture · Desktop Application*

A desktop workbench for manually mapping and documenting software architecture. Built around an architect-defined graph model, Trace represents files, classes, functions, variables, and their relationships without relying on automated code scanning to infer structure.

What I built:

- Implemented a graph state model with UUID-based node and edge identity, semantic relationship types, nested nodes, and directed or bidirectional edges.
- Built a PySide6 canvas with command-driven node creation, layer filtering, zoom, drag, and drop hierarchy management.
- Implemented JSON workspace save/load, preserving node positions, hierarchy, and edge relationships while resolving a Qt parent-child coordinate issue that caused nested nodes to shift after reload.

Why it exists: Working on large projects taught me I need to see where all functions live and how pipelines connect. Scanning gives you the graph. Building it manually gives you understanding of the graph.

Source: github.com/lynekojawa/Project-Trace

Determinant-Engine *Finite Field Mathematics · Algorithm · Linear Algebra*

A determinant calculator built to support matrix computations in finite field. Inspired by computational work during my undergraduate research on ElGamal encryption the project provides an independent implementation for calculating determinants and inspecting intermediate steps.

What I built:

- Implemented a matrix computation engine with dimension validation, base case for (1×1) and (2×2) matrices, and recursive cofactor expansion.
- Replaced factorial-time cofactor expansion with Gaussian elimination to compute determinants in $(O(n^3))$ time.
- Built a Streamlit interface with LaTeX-rendered matrices for determinant calculations.

Why it exists: During undergraduate ElGamal cryptography research, I grew frustrated modifying Mathematica scripts for finite field determinant computations. Accessible tooling did not exist, so I built it.

Source: github.com/lynekojawa/The-Determinant-Engine

Logic-Orchestrator *Multi-Agent Systems · Context Engineering · AI Architecture*

A hierarchical multi-agent system designed to coordinate specialized AI agents across coding, research, career development, and personal workflows. Built around explicit agent responsibilities, versioned prompts, and persistent notes to support continuity across tasks and sessions.

What I built:

- Designed a multi-tier agent hierarchy with specialized roles, separating task coordination from domain-specific work.
- Organized agent configurations and responsibilities through versioned prompt files, a centralized roster and architecture documentation.
- Maintained agent-specific notes and infrastructure records to track context, configuration changes and system stability over time.

Why it exists: I built Logic-Orchestrator to make my way of working with AI more structured and scalable: scalable starting from setting up a structured git workflow and evolving into a multi-agent orchestration. **Source:** github.com/lynekojawa/logic-orchestrator